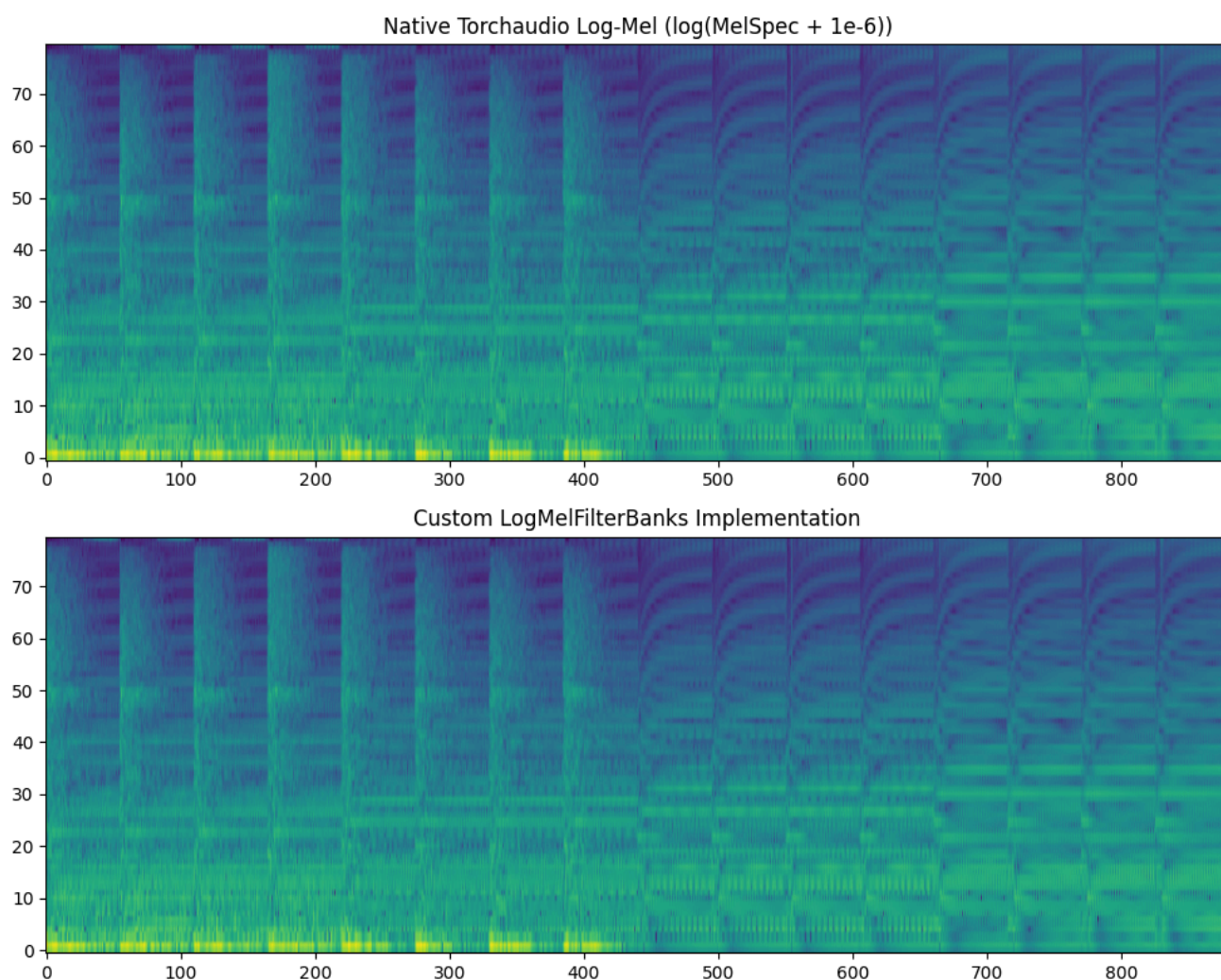


Task report: Digital Signal Processing & CNN Training

Student: Trifonov Sergei

1. Извлечение признаков (Log Mel-scale Filterbank)

Был реализован кастомный слой `LogMelFilterBanks` на чистом PyTorch для извлечения логарифмов энергий мел-спектрограммы. Результат работы кастомного слоя был успешно сверен с нативной реализацией `torchaudio.transforms.MelSpectrogram` (с добавлением логарифмирования). Различий в тензорах не выявлено.

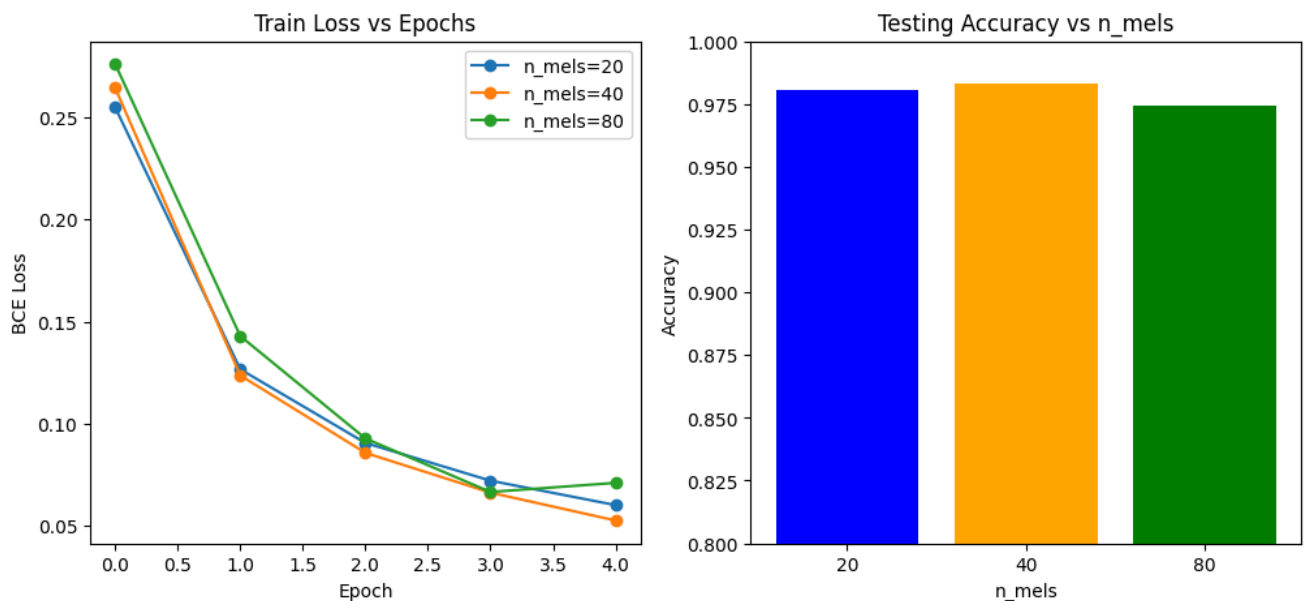


2. Эксперимент с количеством мел-фильтров (`n_mels`)

Для бинарной классификации датасета Speech Commands ("yes" / "no") была построена простая сверточная нейронная сеть (1D CNN). В первом эксперименте варьировался параметр `n_mels` (количество фильтров в мел-спектрограмме) для оценки его влияния на размер модели, вычислительную сложность и точность.

Результаты эксперимента:

n_mels	Params (Кол-во параметров)	FLOPs (Вычислит. сложность)	Test Accuracy (Точность на тесте)
20	20.9K	1.7M	0.9806 (98.06%)
40	22.8K	2.1M	0.9830 (98.30%)
80	26.7K	2.8M	0.9745 (97.45%)



Выводы:

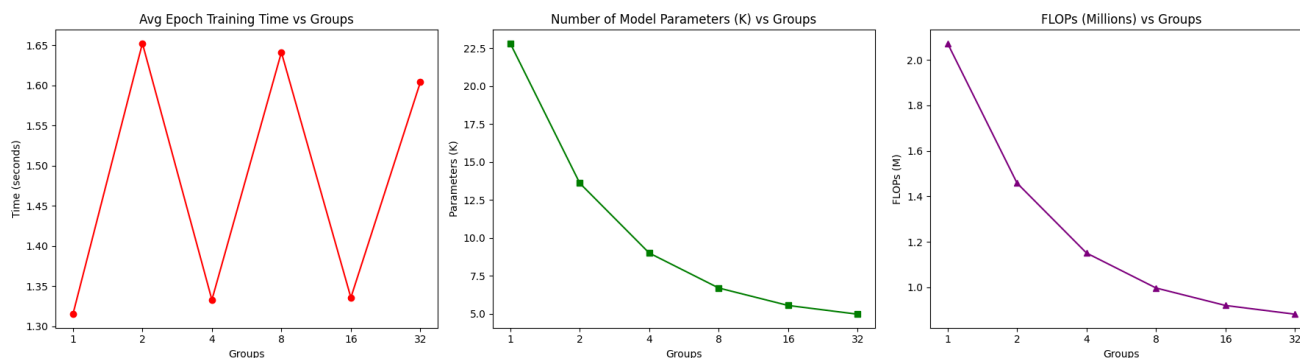
- **Размер и сложность:** С увеличением `n_mels` линейно возрастает количество параметров модели (от 20.9K до 26.7K) и количество математических операций (FLOPs). Это логично, так как увеличивается размерность входного тензора для первого сверточного слоя.
- **Качество (Ассигасу):** Наилучший результат показала модель с `n_mels = 40` (98.30%). Использование всего 20 фильтров дает хорошую точность, но, вероятно, теряет часть важных частотных деталей. Переход к 80 фильтрам привел к небольшому падению точности (до 97.45%). Это может быть связано с тем, что для небольшой модели (менее 30K параметров) слишком высокое разрешение входных признаков приводит к переобучению (overfitting) или модель не обладает достаточной емкостью, чтобы эффективно обработать такое количество каналов.
- **Бейзлайн:** Для следующего эксперимента в качестве оптимального бейзлайна была выбрана модель с `n_mels = 40`.

3. Эксперимент с групповыми свертками (groups)

В данном эксперименте параметр `groups` в слоях `Conv1d` изменялся в диапазоне {1, 2, 4, 8, 16, 32}. Базовая размерность признаков `n_mels = 40`.

Результаты эксперимента (метрики на 5-й эпохе):

groups	Train Loss	Validation Accuracy	Среднее время эпохи
1	0.0598	98.51%	~1.32s
2	0.0620	97.38%	~1.65s
4	0.0710	97.26%	~1.33s
8	0.0889	97.26%	~1.64s
16	0.0999	97.38%	~1.34s
32	0.1222	95.77%	~1.60s



Выводы:

- **Влияние на Loss и Accuracy:** Как видно из данных, при увеличении значения `groups` обучающий Loss (ошибка) постепенно возрастает (с 0.0598 до 0.1222), а точность на валидации падает (с 98.51% до 95.77% при `groups=32`). Это подтверждает теорию: при групповых свертках фильтры "видят" только часть входных каналов, из-за чего теряются межканальные взаимосвязи, что снижает выразительную способность (expressiveness) модели.
- **Время обучения:** Несмотря на значительное теоретическое снижение FLOPs, время выполнения одной эпохи не уменьшилось, а в некоторых случаях даже немного колебалось в большую сторону (от 1.3s до 2.1s).
- **Причины отсутствия ускорения:** Это объясняется тем, что наша модель *экстремально маленькая* (всего ~20K параметров). Для современных GPU расчет такой сети происходит практически мгновенно, и узким местом становится не математика, а накладные расходы фреймворка: пересылка данных, запуск CUDA-ядер и загрузка данных DataLoader-ом. Таким образом, для очень маленьких моделей групповые свертки не дают выигрыша во времени на GPU, но отлично подходят для сжатия размера сети при деплое на слабые устройства.